

Guia completa del sistema Red Clubes

Que existe, para que sirve y como entenderlo de principio a fin

Generado el 08/06/2026

Objetivo: que puedas mirar el proyecto sin depender de IA y entender que hace cada carpeta, que hace cada archivo importante y por donde empezar cuando quieras cambiar o agregar una funcion.

1. Mapa mental del sistema

Red Clubes esta dividido en tres piezas principales: una interfaz Angular, una API Java Spring Boot y una base de datos MySQL. La interfaz no guarda datos permanentes; pide y muestra informacion. El backend concentra reglas, permisos y calculos. La base de datos conserva clubes, socios, usuarios, actividades, cuotas y asistencias.

- Frontend: lo que ve y toca el usuario.
- Backend: lo que decide, valida y responde.
- Base de datos: la memoria persistente del sistema.
- Documentacion y scripts: ayudas para entender, instalar y arrancar el proyecto.

2. Como viaja una accion

Cuando el usuario hace clic en una pantalla, Angular ejecuta una funcion en app.ts. Esa funcion llama a una URL del backend. El controlador Java recibe el pedido, el servicio aplica reglas de negocio, el repositorio consulta o guarda en MySQL y luego el backend devuelve JSON. Angular actualiza la pantalla con esa respuesta.

- Ejemplo: guardar un socio empieza en el formulario del frontend.
- La llamada llega a SocioController.
- SocioService valida y decide que guardar.
- SocioRepository persiste la entidad Socio.
- El frontend refresca tarjetas, tablas o mensajes.

3. Carpetas del proyecto

Carpeta	Funcion dentro del sistema
./	Contiene los accesos rapidos, configuracion general y las tres areas reales del sistema: frontend, backend y documentacion.
.github/	Espacio reservado para automatizaciones de GitHub, como acciones de integracion o plantillas del repositorio.
.vscode/	Ajustes locales para abrir el proyecto comodo en Visual Studio Code sin cambiar el codigo del sistema.
backend/	Aplicacion Java Spring Boot. Expone la API, valida reglas de negocio y guarda/lee datos en MySQL.
backend/src/main/java/com/redclubes/backend/	Codigo principal del servidor, dividido por modulos del negocio.
backend/src/main/resources/	Configuracion que usa el servidor al arrancar.
backend/src/test/	Pruebas automaticas del backend.
database/	Scripts o recursos relacionados con la base de datos.
docs/	Documentacion funcional y tecnica del proyecto.
frontend/	Aplicacion Angular que ve el usuario en el navegador.
frontend/public/	Archivos estaticos publicados tal cual, como iconos e imagenes de marca.
frontend/src/	Codigo fuente de la interfaz Angular.
frontend/src/app/	Pantalla principal, configuracion de Angular y estilos especificos de la app.

4. Frontend Angular

El frontend es una aplicacion Angular. Hoy esta concentrada en un componente principal grande: app.ts, app.html y app.css. Para un MVP esto acelera, porque todo el flujo esta cerca. A futuro conviene separar por modulos: socios, actividades, cuotas, asistencias, reportes y configuracion.

- app.ts guarda estado, llama al backend y ejecuta acciones.
- app.html dibuja la interfaz segun el estado actual.
- app.css mantiene el aspecto visual de tarjetas, botones, menu lateral y modales.
- app.config.ts conecta servicios globales de Angular como HTTP.

5. Backend Spring Boot

El backend esta organizado por modulos del negocio. Cada modulo suele tener entidades, repositorios, servicios, controladores y objetos Request/Response. Esa separacion evita mezclar pantalla, reglas y datos.

- Controller: recibe HTTP y responde JSON.
- Service: contiene reglas de negocio.
- Repository: habla con la base de datos.
- Entity: representa una tabla.
- Request/Response: definen contratos limpios con el frontend.

6. Modulos del backend

Modulo	Responsabilidad
config	Configuracion transversal: CORS y manejo global de errores.
health	Chequeo tecnico para saber si el backend esta funcionando.
clubes	Alta, edicion, consulta y estado de clubes.
socios	Gestion de socios/personas asociadas a un club.
gestion	Operaciones diarias: dashboard, actividades, inscripciones, cuotas, asistencias y reportes.
usuarios	Login, sesiones, perfiles, roles y permisos por club o actividad.

7. Modelo de datos explicado

Entidad	Que representa	Relaciones clave
Club	Una institucion dentro de la red.	Tiene socios, actividades, usuarios asignados, cuotas y asistencias.
Usuario	Persona que entra al sistema.	Puede tener rol global, clubes asignados y actividades asignadas.
Socio	Persona asociada a un club.	Pertenece a un club y puede inscribirse a actividades.
Actividad	Disciplina, taller o actividad del club.	Pertenece a un club y recibe inscripciones/asistencias.
InscripcionActividad	Union entre socio y actividad.	Permite saber que socio participa en que actividad.
Cuota	Pago o deuda de un socio.	Pertenece a socio y club; tiene estado como pagada o pendiente.
Asistencia	Presencia de un socio en una fecha.	Conecta socio, actividad, club y fecha.
SesionUsuario	Sesion activa de login.	Permite autenticar pedidos posteriores al login.
UsuarioClub	Permiso de usuario sobre un club.	Habilita trabajar en determinados clubes.
UsuarioActividad	Permiso de usuario sobre una actividad.	Acota acceso a actividades especificas.

8. Flujos principales

- Login: el usuario envia email y contrasena; el backend valida y devuelve una sesion.
- Seleccion de club: la interfaz usa el club activo para filtrar dashboard, socios, actividades y pagos.
- Dashboard: GestionService calcula tarjetas de resumen, cuotas, morosos, actividades y datos generales.
- Socios: SocioController y SocioService administran altas, ediciones, bajas logicas e inscripciones.
- Actividades: GestionController/GestionService administran disciplinas y participantes.
- Cuotas y pagos: el backend calcula importes, estados y resúmenes; la interfaz muestra deuda/cobro.
- Asistencias: se registran presencias por actividad y fecha.
- Usuarios y roles: usuarios controla quien puede entrar y que parte del sistema puede operar.

9. Inventario de archivos

Archivo	Por que existe / funcion
.github/java-upgrade/.gitignore	Archivo de soporte de herramientas de modernizacion Java; evita guardar salidas temporales de ese flujo.
.github/java-upgrade/hooks/scripts/recordToolUse.ps1	Script PowerShell usado por herramientas de modernizacion para registrar uso de comandos.
.github/java-upgrade/hooks/scripts/recordToolUse.sh	Script shell usado por herramientas de modernizacion para registrar uso de comandos.
.github/modernize/java-upgrade/.gitignore	Archivo de soporte de otra ruta de modernizacion Java; evita versionar salidas temporales.
.github/modernize/java-upgrade/hooks/scripts/recordToolUse.ps1	Script PowerShell de soporte para registrar actividad de herramientas de modernizacion.
.github/modernize/java-upgrade/hooks/scripts/recordToolUse.sh	Script shell de soporte para registrar actividad de herramientas de modernizacion.
.gitignore	Le dice a Git que no versionar: dependencias, compilados, archivos temporales y salidas locales.
.vscode/settings.json	Ajustes recomendados del editor para este workspace.
README.md	Puerta de entrada del proyecto. Resume que es Red Clubes, sus modulos y la tecnologia usada.
backend/.gitattributes	Normaliza atributos de Git dentro del backend, como finales de linea.
backend/.gitignore	Evita versionar compilados, logs y archivos locales del backend.
backend/.mvn/wrapper/maven-wrapper.properties	Configura la version de Maven que usan mvnw y mvnw.cmd.
backend/HELP.md	Documento generado por Spring Initializr con referencias iniciales del backend.
backend/mvnw	Wrapper de Maven para Linux/macOS. Permite compilar sin instalar Maven globalmente.
backend/mvnw.cmd	Wrapper de Maven para Windows.
backend/pom.xml	Configuracion Maven del backend: dependencias Spring Boot, JPA, MySQL, validacion y tests.
backend/src/main/java/com/redclubes/backend/BackendApplication.java	Clase principal. Arranca Spring Boot y carga todos los componentes del servidor.
backend/src/main/java/com/redclubes/backend/clubes/ActualizarClubRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/clubes/Club.java	Entidad JPA del modulo backend. Representa una tabla de la base de datos.
backend/src/main/java/com/redclubes/backend/clubes/ClubController.java	Controlador HTTP del modulo backend. Recibe pedidos del frontend y devuelve respuestas JSON.
backend/src/main/java/com/redclubes/backend/clubes/ClubInitializer.java	Inicializador del modulo backend. Crea datos iniciales o demo al arrancar si hacen falta.
backend/src/main/java/com/redclubes/backend/clubes/ClubNoEncontradoException.java	Error especifico del modulo backend. Ayuda a responder con mensajes claros.
backend/src/main/java/com/redclubes/backend/clubes/ClubRepository.java	Repositorio JPA del modulo backend. Lee y escribe entidades en MySQL.
backend/src/main/java/com/redclubes/backend/clubes/ClubResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/clubes/ClubService.java	Servicio del modulo backend. Contiene reglas de negocio y coordina repositorios.
backend/src/main/java/com/redclubes/backend/clubes/CrearClubRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/clubes/EstadoClub.java	Enumeracion del modulo backend. Limita valores posibles para evitar estados escritos a mano.
backend/src/main/java/com/redclubes/backend/config/CorsConfig.java	Permite que el frontend Angular pueda llamar al backend desde otro puerto durante desarrollo.

Archivo	Por que existe / funcion
backend/src/main/java/com/redclubes/backend/config/GlobalExceptionHandler.java	Convierte errores del backend en respuestas HTTP claras para el frontend.
backend/src/main/java/com/redclubes/backend/gestion/Actividad.java	Entidad JPA del modulo backend. Representa una tabla de la base de datos.
backend/src/main/java/com/redclubes/backend/gestion/ActividadRepository.java	Repositorio JPA del modulo backend. Lee y escribe entidades en MySQL.
backend/src/main/java/com/redclubes/backend/gestion/ActividadResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/gestion/ActualizarActividadRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/gestion/ActualizarInscripcionesRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/gestion/Asistencia.java	Entidad JPA del modulo backend. Representa una tabla de la base de datos.
backend/src/main/java/com/redclubes/backend/gestion/AsistenciaRepository.java	Repositorio JPA del modulo backend. Lee y escribe entidades en MySQL.
backend/src/main/java/com/redclubes/backend/gestion/AsistenciaResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/gestion/CrearActividadRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/gestion/Cuota.java	Entidad JPA del modulo backend. Representa una tabla de la base de datos.
backend/src/main/java/com/redclubes/backend/gestion/CuotaRepository.java	Repositorio JPA del modulo backend. Lee y escribe entidades en MySQL.
backend/src/main/java/com/redclubes/backend/gestion/CuotaResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/gestion/DashboardResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/gestion/DatosDemoInitializer.java	Inicializador del modulo backend. Crea datos iniciales o demo al arrancar si hacen falta.
backend/src/main/java/com/redclubes/backend/gestion/EstadoActividad.java	Enumeracion del modulo backend. Limita valores posibles para evitar estados escritos a mano.
backend/src/main/java/com/redclubes/backend/gestion/EstadoCuota.java	Enumeracion del modulo backend. Limita valores posibles para evitar estados escritos a mano.
backend/src/main/java/com/redclubes/backend/gestion/EstadoInscripcion.java	Enumeracion del modulo backend. Limita valores posibles para evitar estados escritos a mano.
backend/src/main/java/com/redclubes/backend/gestion/GestionController.java	Controlador HTTP del modulo backend. Recibe pedidos del frontend y devuelve respuestas JSON.
backend/src/main/java/com/redclubes/backend/gestion/GestionService.java	Servicio del modulo backend. Contiene reglas de negocio y coordina repositorios.
backend/src/main/java/com/redclubes/backend/gestion/GraficoMesResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/gestion/GuardarAsistenciaRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/gestion/InscripcionActividad.java	Entidad JPA del modulo backend. Representa una tabla de la base de datos.
backend/src/main/java/com/redclubes/backend/gestion/InscripcionActividadRepository.java	Repositorio JPA del modulo backend. Lee y escribe entidades en MySQL.
backend/src/main/java/com/redclubes/backend/gestion/MovimientoResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/gestion/ReportesResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.

Archivo	Por que existe / funcion
backend/src/main/java/com/redclubes/backend/gestion/ResumenClubResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/gestion/SocioResumenResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/health/HealthController.java	Endpoint simple para comprobar que el backend esta vivo.
backend/src/main/java/com/redclubes/backend/socios/Socio.java	Entidad JPA del modulo backend. Representa una tabla de la base de datos.
backend/src/main/java/com/redclubes/backend/socios/SocioController.java	Controlador HTTP del modulo backend. Recibe pedidos del frontend y devuelve respuestas JSON.
backend/src/main/java/com/redclubes/backend/socios/SocioNoEncontradoException.java	Error especifico del modulo backend. Ayuda a responder con mensajes claros.
backend/src/main/java/com/redclubes/backend/socios/SocioRepository.java	Repositorio JPA del modulo backend. Lee y escribe entidades en MySQL.
backend/src/main/java/com/redclubes/backend/socios/SocioService.java	Servicio del modulo backend. Contiene reglas de negocio y coordina repositorios.
backend/src/main/java/com/redclubes/backend/usuarios/AccesoDenegadoException.java	Error especifico del modulo backend. Ayuda a responder con mensajes claros.
backend/src/main/java/com/redclubes/backend/usuarios/ActualizarAsignacionesUsuarioRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/usuarios/ActualizarPerfilRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/usuarios/AsignacionUsuarioRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/usuarios/AuthController.java	Controlador HTTP del modulo backend. Recibe pedidos del frontend y devuelve respuestas JSON.
backend/src/main/java/com/redclubes/backend/usuarios/AuthService.java	Servicio del modulo backend. Contiene reglas de negocio y coordina repositorios.
backend/src/main/java/com/redclubes/backend/usuarios/CambiarPasswordRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/usuarios/ClubAsignadoResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/usuarios/CrearUsuarioRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/usuarios/CredencialesInvalidasException.java	Error especifico del modulo backend. Ayuda a responder con mensajes claros.
backend/src/main/java/com/redclubes/backend/usuarios/EstadoUsuario.java	Enumeracion del modulo backend. Limita valores posibles para evitar estados escritos a mano.
backend/src/main/java/com/redclubes/backend/usuarios/LoginRequest.java	Objeto de entrada del modulo backend. Define los datos que el frontend envia al backend.
backend/src/main/java/com/redclubes/backend/usuarios/LoginResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/usuarios/PasswordService.java	Servicio del modulo backend. Contiene reglas de negocio y coordina repositorios.
backend/src/main/java/com/redclubes/backend/usuarios/RolClub.java	Enumeracion del modulo backend. Limita valores posibles para evitar estados escritos a mano.
backend/src/main/java/com/redclubes/backend/usuarios/RolUsuario.java	Enumeracion del modulo backend. Limita valores posibles para evitar estados escritos a mano.
backend/src/main/java/com/redclubes/backend/usuarios/SesionUsuario.java	Entidad JPA del modulo backend. Representa una tabla de la base de datos.
backend/src/main/java/com/redclubes/backend/usuarios/SesionUsuarioRepository.java	Repositorio JPA del modulo backend. Lee y escribe entidades en MySQL.

Archivo	Por que existe / funcion
backend/src/main/java/com/redclubes/backend/usuarios/SuperUsuarioInitializer.java	Inicializador del modulo backend. Crea datos iniciales o demo al arrancar si hacen falta.
backend/src/main/java/com/redclubes/backend/usuarios/Usuario.java	Entidad JPA del modulo backend. Representa una tabla de la base de datos.
backend/src/main/java/com/redclubes/backend/usuarios/UsuarioActividad.java	Entidad JPA del modulo backend. Representa una tabla de la base de datos.
backend/src/main/java/com/redclubes/backend/usuarios/UsuarioActividadRepository.java	Repositorio JPA del modulo backend. Lee y escribe entidades en MySQL.
backend/src/main/java/com/redclubes/backend/usuarios/UsuarioClub.java	Entidad JPA del modulo backend. Representa una tabla de la base de datos.
backend/src/main/java/com/redclubes/backend/usuarios/UsuarioClubRepository.java	Repositorio JPA del modulo backend. Lee y escribe entidades en MySQL.
backend/src/main/java/com/redclubes/backend/usuarios/UsuarioController.java	Controlador HTTP del modulo backend. Recibe pedidos del frontend y devuelve respuestas JSON.
backend/src/main/java/com/redclubes/backend/usuarios/UsuarioNoEncontradoException.java	Error especifico del modulo backend. Ayuda a responder con mensajes claros.
backend/src/main/java/com/redclubes/backend/usuarios/UsuarioRepository.java	Repositorio JPA del modulo backend. Lee y escribe entidades en MySQL.
backend/src/main/java/com/redclubes/backend/usuarios/UsuarioResponse.java	Objeto de salida del modulo backend. Define los datos que el backend devuelve al frontend.
backend/src/main/java/com/redclubes/backend/usuarios/UsuarioService.java	Servicio del modulo backend. Contiene reglas de negocio y coordina repositorios.
backend/src/main/resources/application.properties	Configuracion de puerto, base de datos, JPA y comportamiento de arranque del backend.
backend/src/test/java/com/redclubes/backend/BackendApplicationTests.java	Prueba que valida que el contexto Spring puede arrancar.
database/.gitkeep	Mantiene la carpeta database dentro de Git aunque todavia no tenga scripts reales.
docs/alcance-mvp.md	Define que incluye la primera version util del producto y que queda fuera.
docs/arquitectura.md	Describe la arquitectura general: navegador, API Java y base MySQL.
docs/modelo-datos.md	Explica las entidades del dominio y como se relacionan.
frontend/.editorconfig	Reglas de formato compartidas para editores: indentacion, charset y finales de linea.
frontend/.gitignore	Evita versionar dependencias, compilados y archivos locales del frontend.
frontend/.prettierrc	Configuracion del formateador Prettier para mantener estilo consistente.
frontend/.vscode/extensions.json	Recomienda extensiones de VS Code utiles para trabajar en Angular.
frontend/.vscode/launch.json	Configuracion de depuracion del frontend en VS Code.
frontend/.vscode/mcp.json	Configuracion local de herramientas MCP para el entorno de desarrollo.
frontend/.vscode/tasks.json	Tareas de VS Code para ejecutar acciones frecuentes del frontend.
frontend/README.md	Documentacion generada por Angular con comandos basicos del frontend.
frontend/angular.json	Configuracion de Angular CLI: build, serve, tests, estilos y assets.
frontend/package-lock.json	Congela versiones exactas de dependencias para instalaciones repetibles.
frontend/package.json	Lista dependencias del frontend y comandos como start, build y test.
frontend/public/brand/red-clubes-logo.png	Logo completo usado en la cabecera de la aplicacion.
frontend/public/brand/red-clubes-mark.png	Marca compacta usada cuando se necesita solo el simbolo.
frontend/public/favicon.ico	Icono del navegador.
frontend/src/app/app.config.ts	Configura proveedores globales de Angular, incluyendo HTTP y animaciones/rutas si aplican.

Archivo	Por que existe / funcion
frontend/src/app/app.css	Estilos especificos de la app: layout, tarjetas, sidebar, botones, tablas, modales e iconos Lucide.
frontend/src/app/app.html	Plantilla visual principal. Muestra menu, dashboard, tarjetas, tablas, formularios y modales.
frontend/src/app/app.routes.ts	Declara las rutas de Angular. Hoy la app funciona principalmente desde una pantalla central.
frontend/src/app/app.spec.ts	Prueba base del componente principal.
frontend/src/app/app.ts	Cerebro actual de la interfaz. Maneja estado, llamadas HTTP, sesion, filtros, formularios y acciones de pantalla.
frontend/src/index.html	HTML base donde Angular monta la aplicacion.
frontend/src/main.ts	Punto de arranque del frontend. Carga el componente raiz con la configuracion de Angular.
frontend/src/styles.css	Estilos globales compartidos por toda la interfaz.
frontend/tsconfig.app.json	Configuracion TypeScript usada para compilar la app.
frontend/tsconfig.json	Configuracion base de TypeScript para el proyecto Angular.
frontend/tsconfig.spec.json	Configuracion TypeScript usada por las pruebas del frontend.
iniciar-backend.cmd	Atajo para levantar solo el servidor Java.
iniciar-frontend.cmd	Atajo para levantar solo la interfaz Angular.
iniciar-red-clubes.cmd	Atajo para iniciar el sistema completo desde Windows.

10. Archivos generados y locales

En el workspace tambien pueden aparecer caches y logs, por ejemplo frontend/.npm-cache, backend/backend.log, frontend/frontend-dev.err.log y frontend/frontend-dev.out.log. No son parte del codigo fuente del producto: los crean npm, el servidor o los scripts de arranque mientras trabajas. Sirven para diagnosticar o acelerar comandos, pero no necesitas estudiarlos para entender el sistema.

Si estos archivos no estan versionados en Git, normalmente se pueden borrar y se vuelven a generar cuando haga falta. Si alguna vez quieres limpiar el proyecto, revisa primero el estado de Git para no eliminar cambios reales.

11. Como agregar una funcion nueva

Usa este orden cuando quieras construir algo por tu cuenta:

- Define que dato nuevo existe o que dato viejo cambia.
- Si necesita guardarse, crea o modifica una entidad JPA.
- Agrega metodos de repositorio solo si necesitas consultas nuevas.
- Pon la regla en el Service, no en el Controller.
- Expone una URL en el Controller usando Request y Response claros.
- En Angular, agrega estado y llamadas HTTP en app.ts.
- Muestra la informacion en app.html y ajusta estilos en app.css.
- Prueba build del frontend y tests del backend.

12. Como saber si un archivo sobra

Un archivo puede ser util aunque no se ejecute directamente. README, docs, scripts, configuraciones y tests sostienen el proyecto. Para borrar algo con seguridad conviene comprobar tres cosas: que no se importe, que no lo use el framework por convencion y que la compilacion/pruebas sigan pasando.

- No borres entidades, DTOs o repositorios solo porque no aparecen en pantalla: pueden ser usados por Spring/JPA.
- No borres package-lock.json: asegura versiones repetibles.
- No borres mvnw/mvnw.cmd: permiten compilar sin instalar Maven.
- No borres configuraciones tsconfig/angular.json/pom.xml: son el mapa de build.

- Antes de borrar, busca referencias y ejecuta build/tests.

13. Guia rapida de orientacion

Quiero hacer...	Mira primero
Cambiar una tarjeta o pantalla	frontend/src/app/app.html y frontend/src/app/app.css
Cambiar datos que ve una pantalla	frontend/src/app/app.ts y el endpoint Java que llama.
Agregar regla de negocio	El Service del modulo correspondiente.
Agregar una URL nueva	Controller + Service + Request/Response.
Agregar una tabla nueva	Entidad JPA + Repository + Service.
Cambiar permisos	usuarios/AuthService y entidades de asignacion.
Entender el dashboard	gestion/GestionService y DashboardResponse.
Entender login	usuarios/AuthController, AuthService, PasswordService y SesionUsuario.

Cierre

La idea central es simple: Angular muestra, Spring decide y MySQL recuerda. Si mantienes esa separacion, vas a poder modificar el sistema sin perderte. Cuando dudes, busca el flujo completo: pantalla, llamada HTTP, controlador, servicio, repositorio, entidad y respuesta.